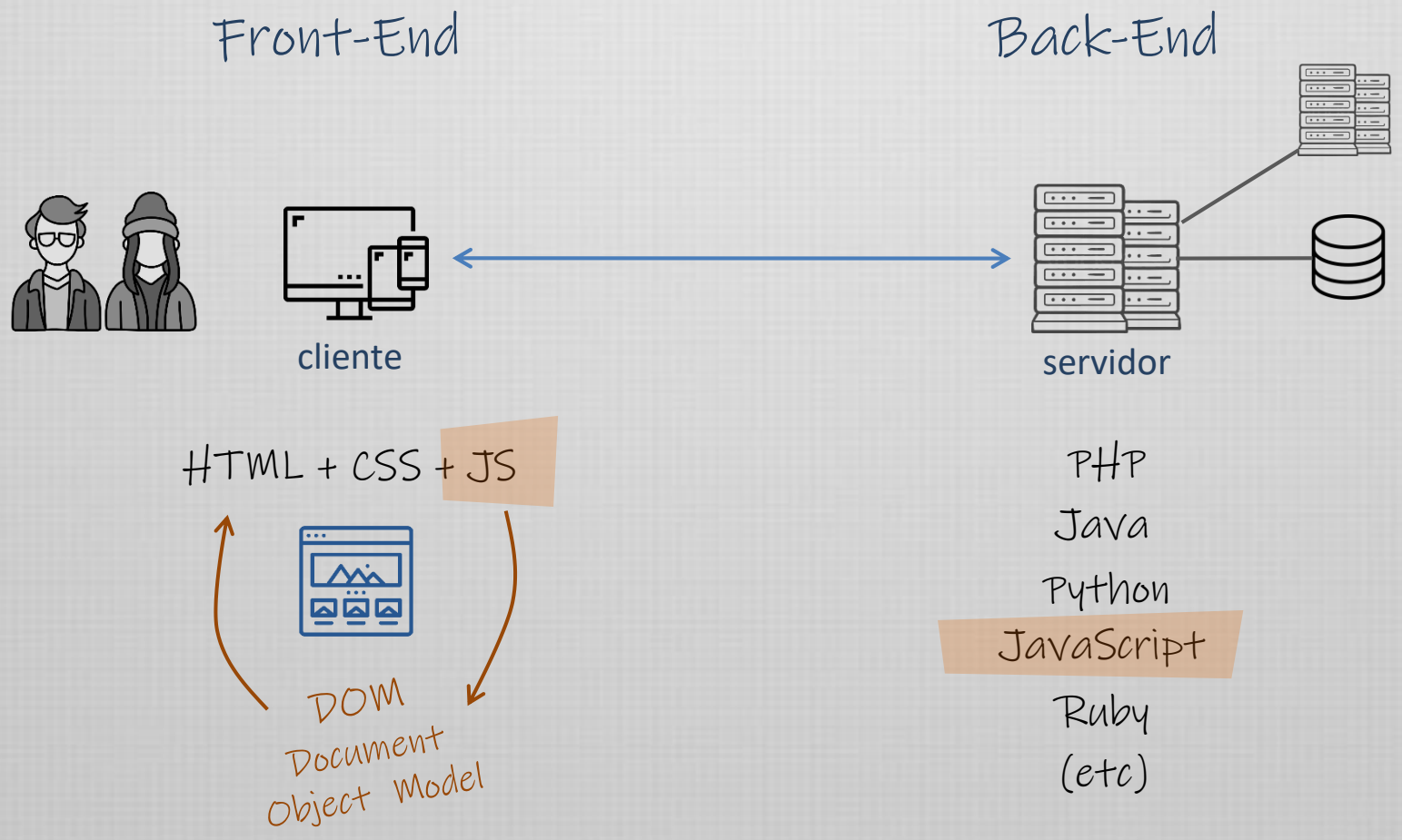
Ingeniería de Aplicaciones Web

Diego C. Martínez

Departamento de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur

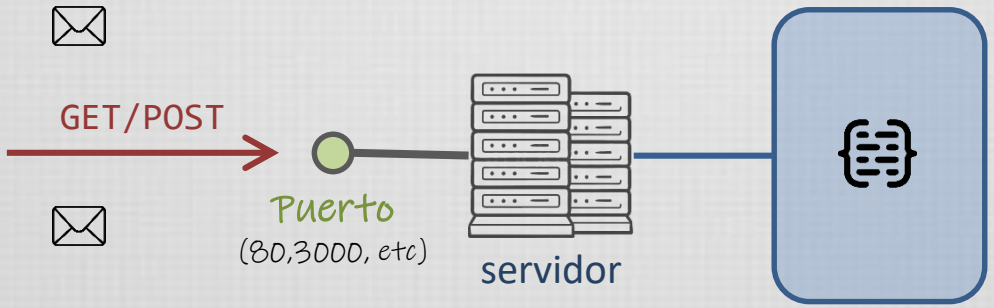
Escenario web



Escenario web

servidor

Escenario web



El servidor está "escuchando" en un Puerto

La **aplicación** reside en el servidor

El servidor ejecuta la aplicación en el **request**

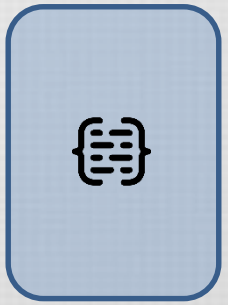
El servidor público es un sistema complejo (Apache, NginX)

Al desarrollar usamos un servidor local liviano

(Ahí parece que el servidor es nuestra aplicación misma, pero está ok por ahora)

No confundir *aplicación web* con *servidor web*

Escenario web



Escenario web

La comunicación hacia la aplicación es básicamente Mensaje HTTP + URL

La aplicación debe responder a cada pedido

GET + URL A	---------->	Realizar acción X
GET + URL B	---------->	Realizar acción Y
POST + URL C	---------->	Realizar acción Z

Todos los frameworks web nos ayudan a especificar esto

Ejemplo de un servidor mínimo en Express

```
const express = require('express')
const app = express()
```

```
app.get('/saludo', (req, res) => {
    res.send('Hola Mundo!')

})

app.get('/upper', (req, res) => {
    res.send(req.query.str.toUpperCase())
})

app.post('/recuperar', (req, res) => {
  const name = req.body.name;
  const email = req.body.email;
  // ... Enviar mail
  res.send('Mail de recuperacion enviado exitosamente');
});
```

```
app.listen(3000,() => console.log('Listening on port 3000'))
```

Ejemplo de un servidor mínimo en Python

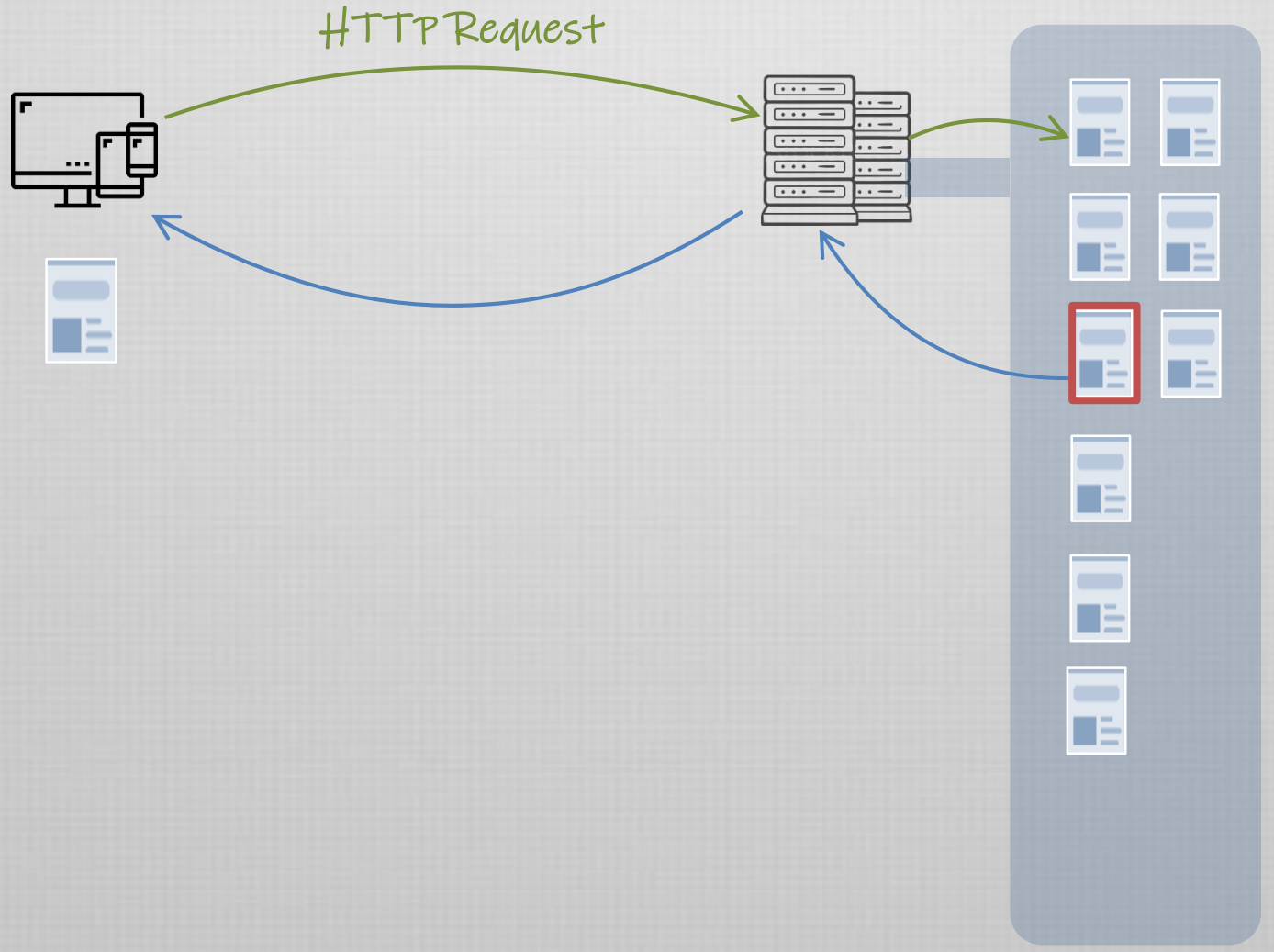
```
from flask import Flask, request

app = Flask(__name__)

@app.route('/saludo')
def saludo():
    return 'Hola Mundo!'

@app.route('/upper')
def upper():
    return request.args.get('str', '').upper()

@app.route('/recuperar', methods=['POST'])
def recuperar():
    name = request.json.get('name')
    email = request.json.get('email')
    # ... Enviar mail
    return 'Mail de recuperacion enviado exitosamente'

if __name__ == '__main__':
    app.run(port=3000)
```

El mismo ejemplo en Laravel

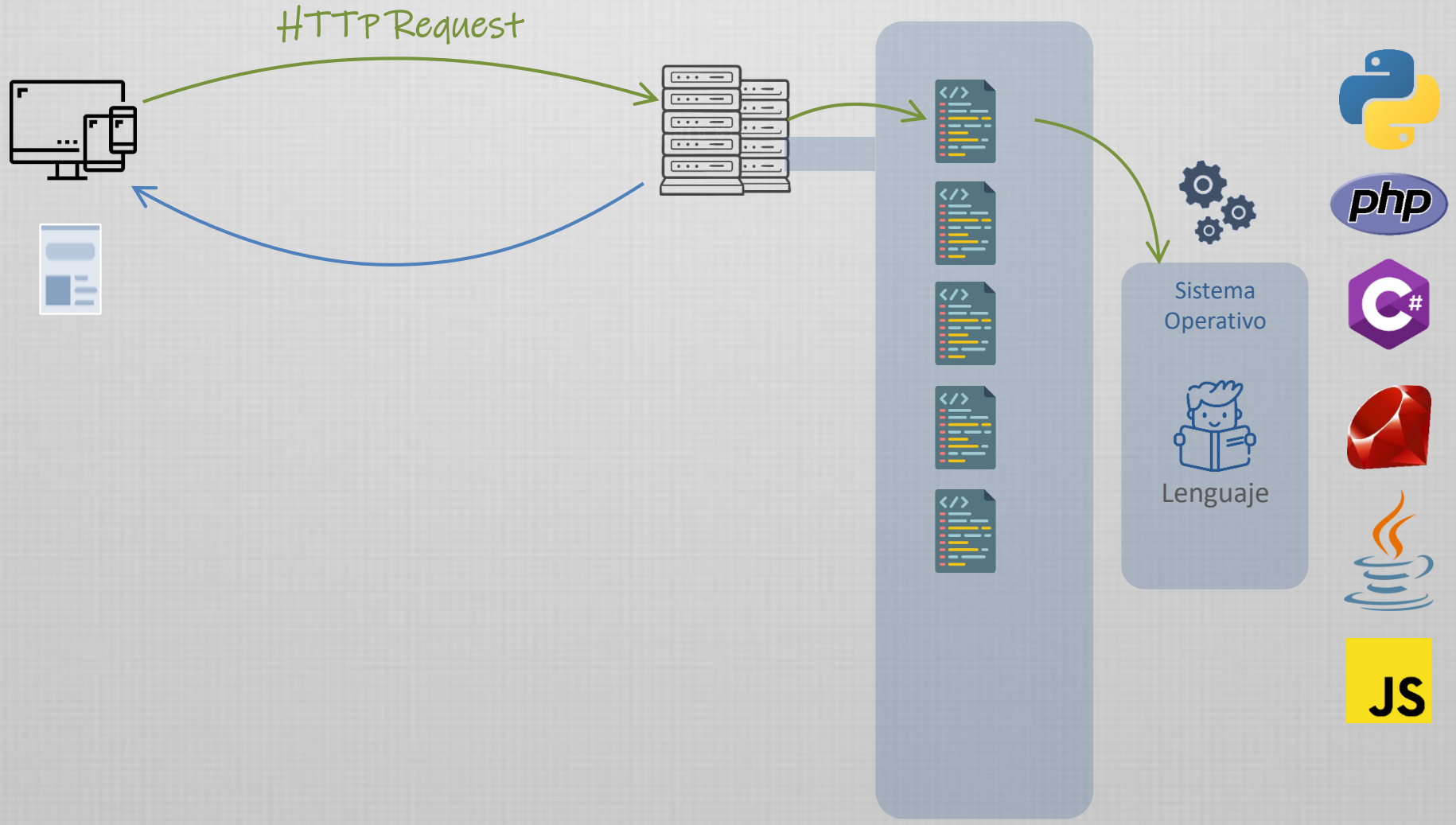
```php
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\AppController;

Route::get('/saludo', [AppController::class, 'saludo']);
Route::get('/upper', [AppController::class, 'upper']);
Route::post('/recuperar', [AppController::class, 'recuperar']);
```

```php
<?php
...
class AppController extends Controller {
    public function upper(Request $request)
    { return strtoupper($request->query('str', '')); }

    public function recuperar(Request $request)
    {   $name = $request->input('name');
        $email = $request->input('email');
        // ... Enviar mail
        return 'Mail de recuperacion enviado exitosamente';
    }
...
```

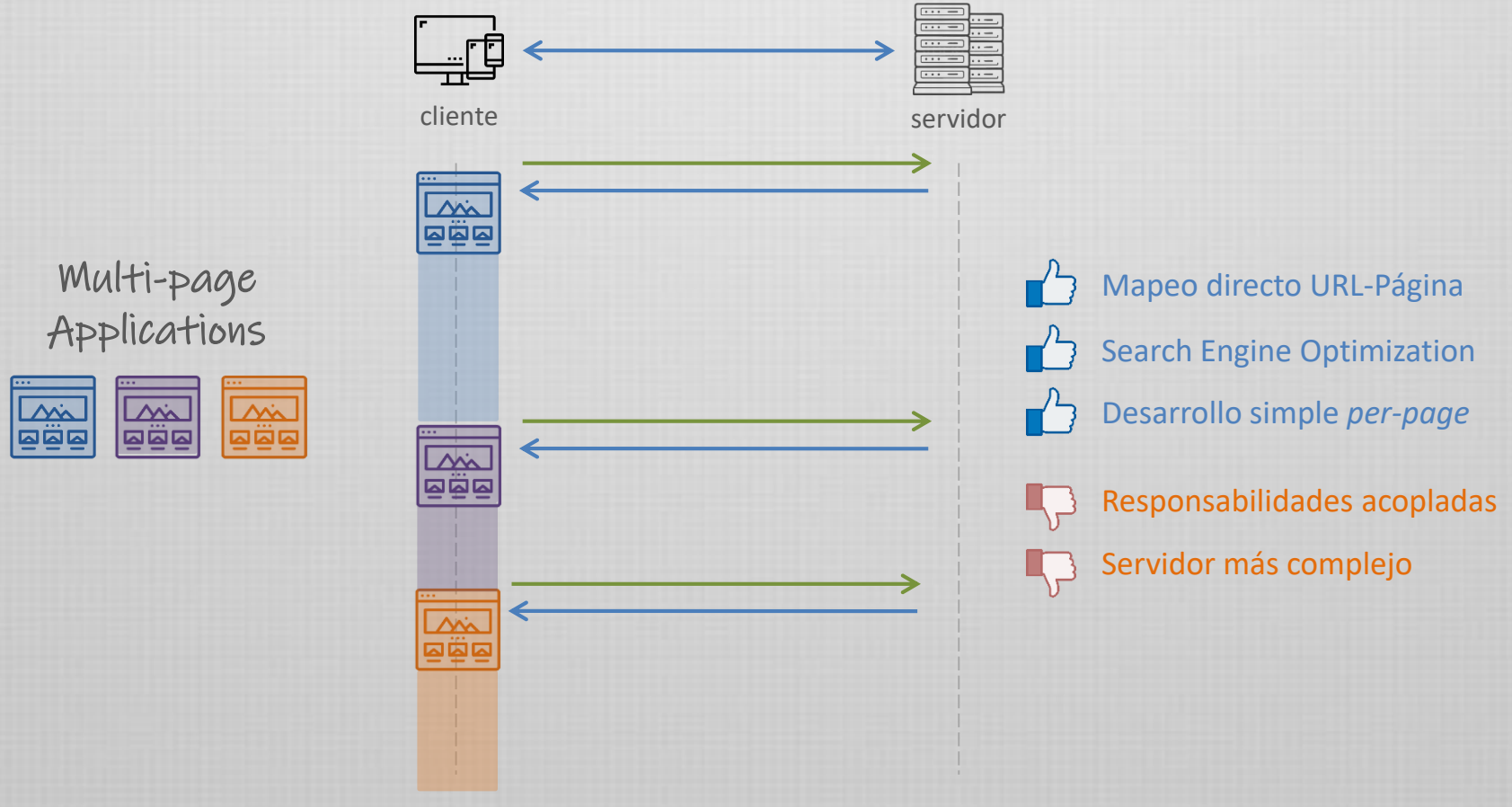
Servidores



Servidores

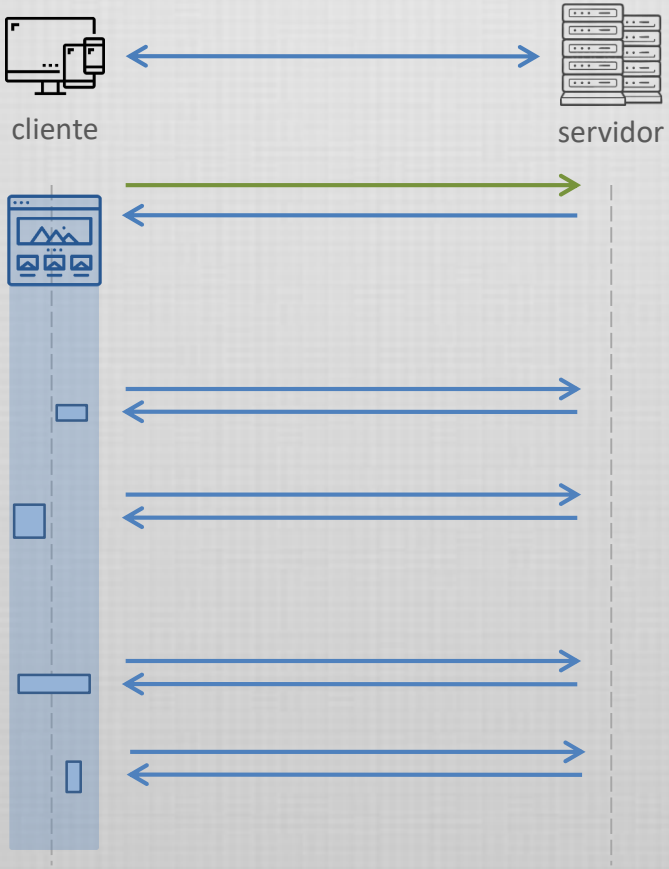


Rendering


cliente
servidor
Multi-page Applications
Mapeo directo URL-Página
Search Engine Optimization
Desarrollo simple *per-page*
Responsabilidades acopladas
Servidor más complejo

Rendering

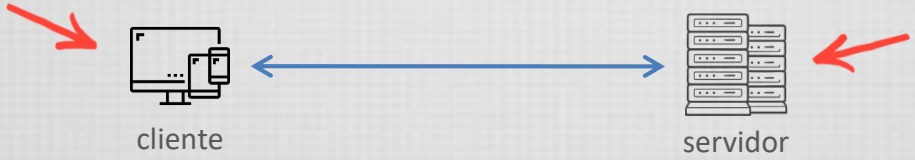
Single-page Applications



- Mejor UX
- Más rápida (bien hecha)
- Aplicaciones móviles

- Seguridad
- Search Engine Optimization

Rendering



En términos generales, la cuestión es quién genera la página que se verá en el cliente

renderiza

PAGE RENDERING

(no confundir con el término "renderizado" en el contexto de los navegadores)

Rendering

Lo que suele resultar confuso es que en ambos casos habrá una página en el cliente que alguien entregó

La diferencia es que esa página puede ser la página "final" o el punto de partida de la generación de una página

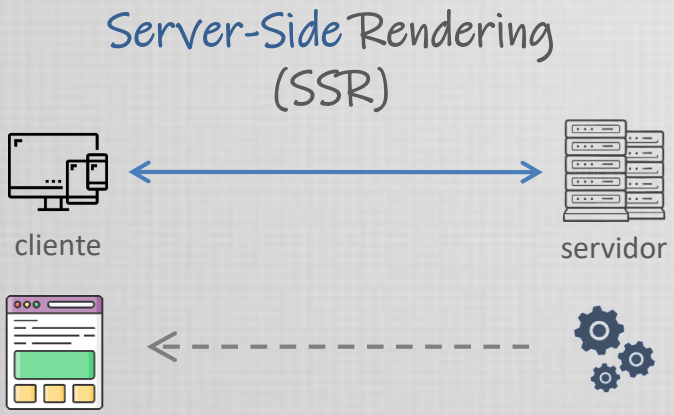
renderizada en el servidor

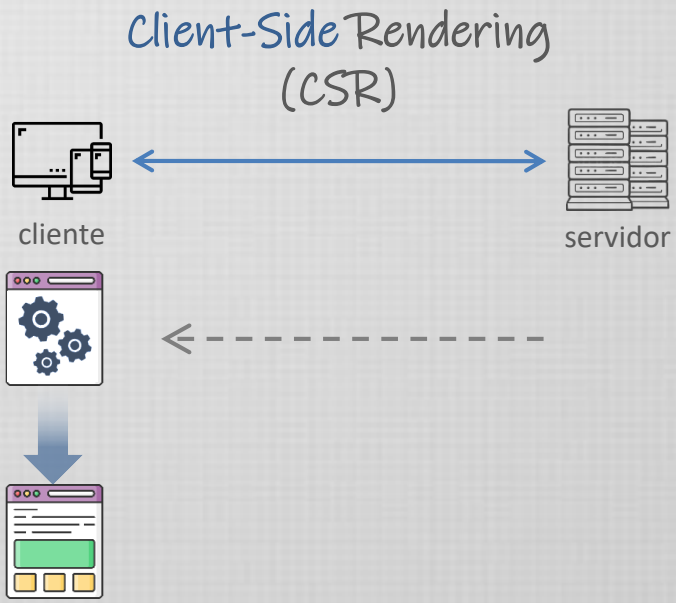
Server-Side Rendering (SSR)

renderizada en el cliente

Client-Side Rendering (CSR)

Rendering

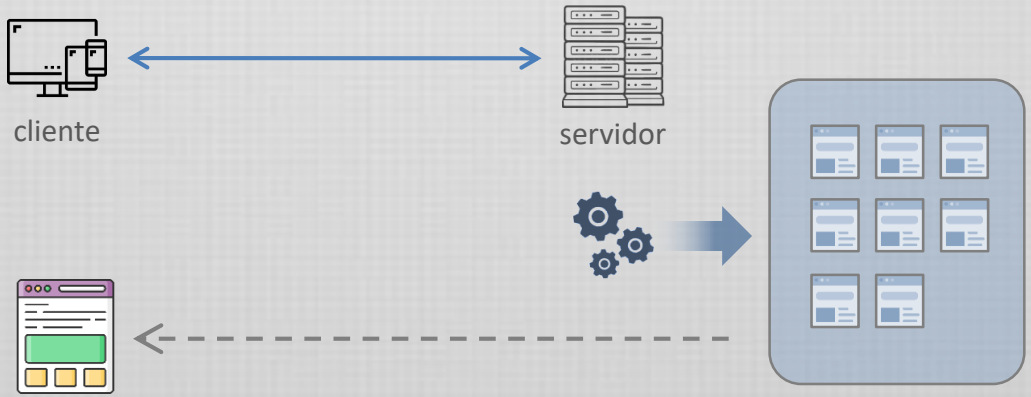




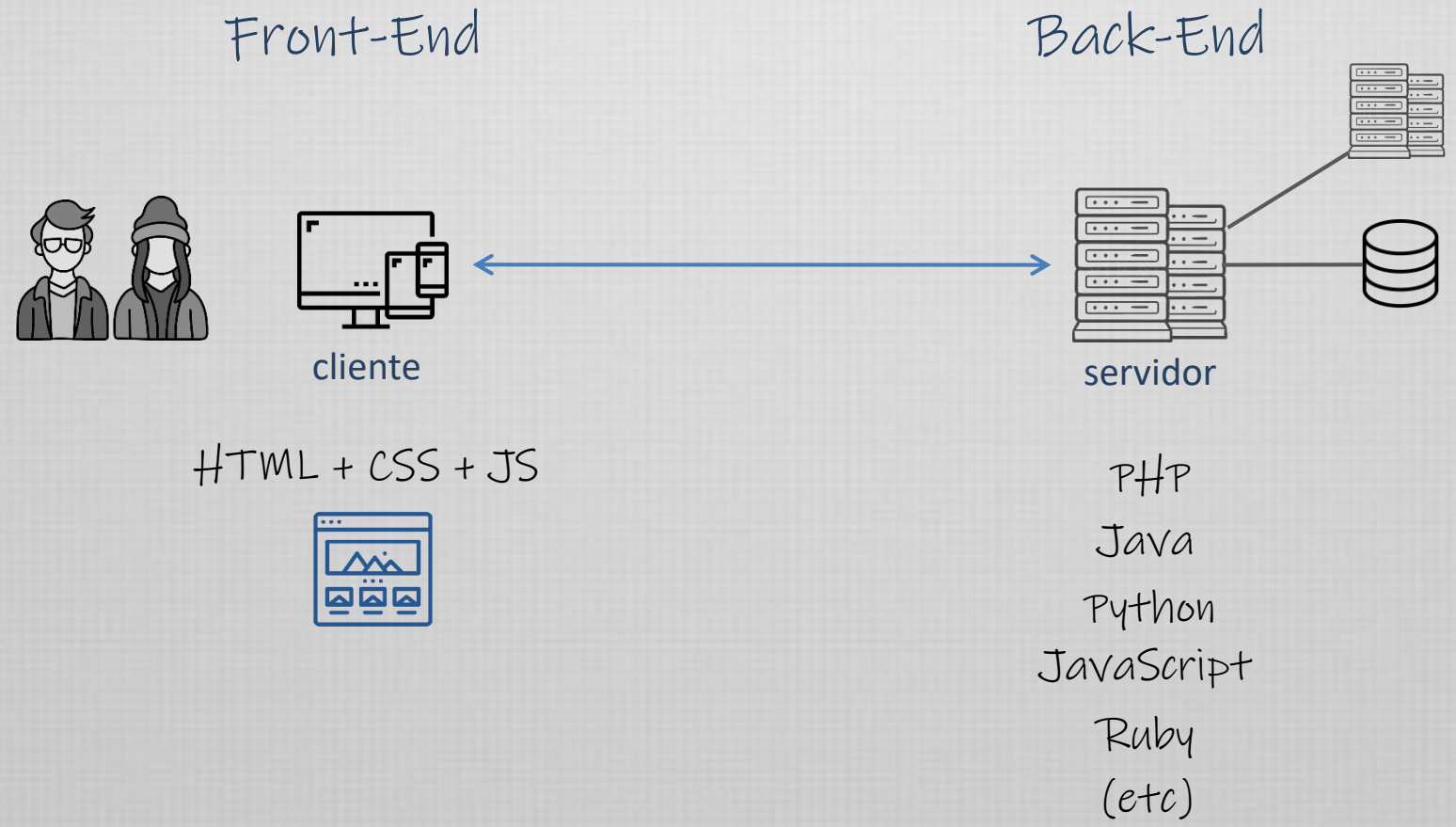
Existen tecnologías diferenciadas para cada esquema

También existen tecnologías que permiten desarrollar con los dos esquemas en convivencia

Static site Generation (SSG)



Escenario web

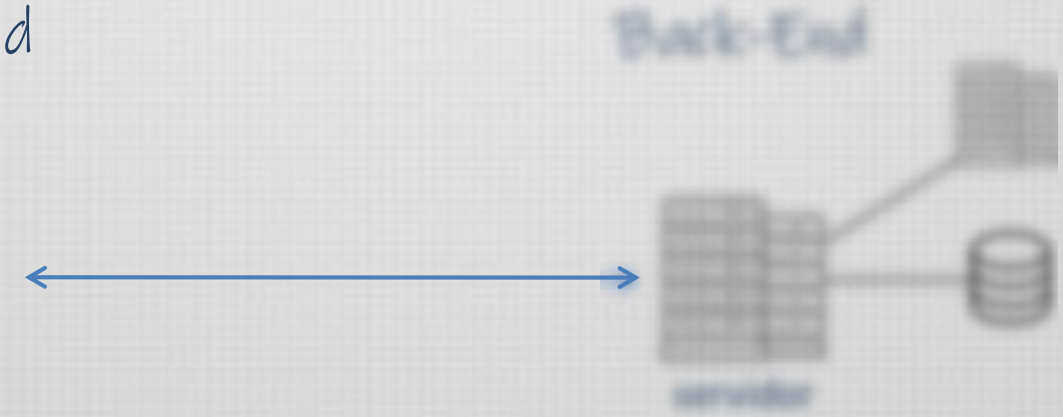


Escenario web

Front-End

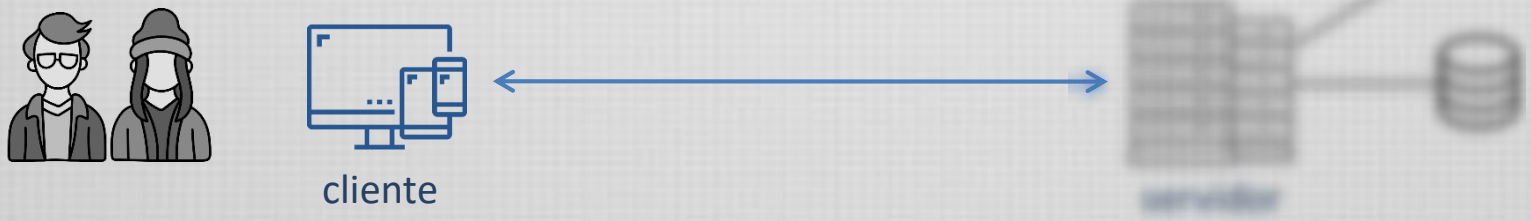
cliente

HTML + CSS + JS



Escenario web

Front-End



Cada vez mas "App"

Adaptable a diferentes pantallas

Más información

Mayor interacción

Desarrollada por equipos

Lenguajes

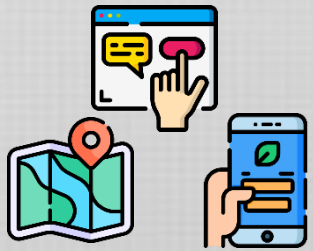
JS

+ tipos estáticos

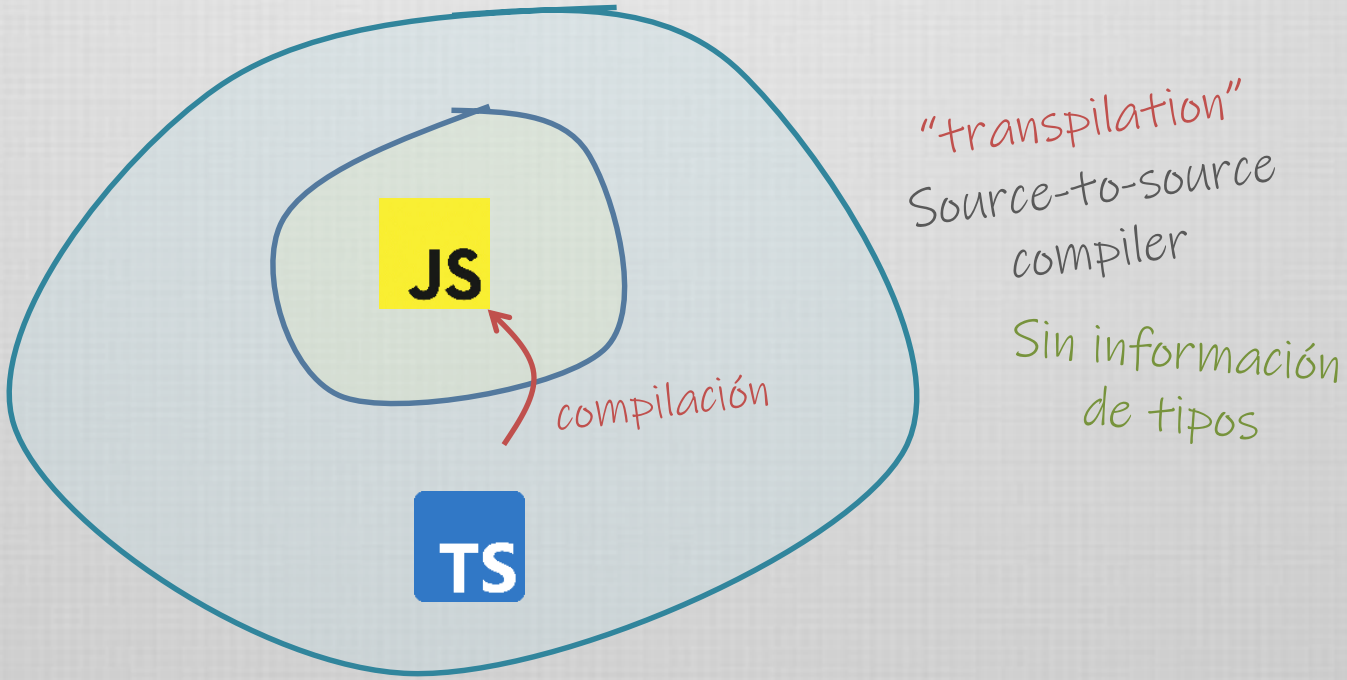
TS

Interacciones simples

Interfaces complejas



Lenguajes



TypeScript nunca cambia el comportamiento en tiempo de ejecución de JavaScript

TypeScript agrega principalmente tipos de datos estáticos

OOP, Duck Typing, Java-style

```
let num: number = 42
let flag: boolean = false

let mensaje: string = 'Hola Mundo'
let saludo: string = `${hello} ${world}!`
```

```typescript
type TimingEvent = { name: "start"; userStarted: boolean } |
                   { name: "closed"; duration: number };
```

```typescript
switch (event.name) {
    case "start":
        const initiatedByUser = event.userStarted;
        break;

    case "closed":
        const timespan = event.duration;
        break;
  }
```

CFA
Control Flow Analysis

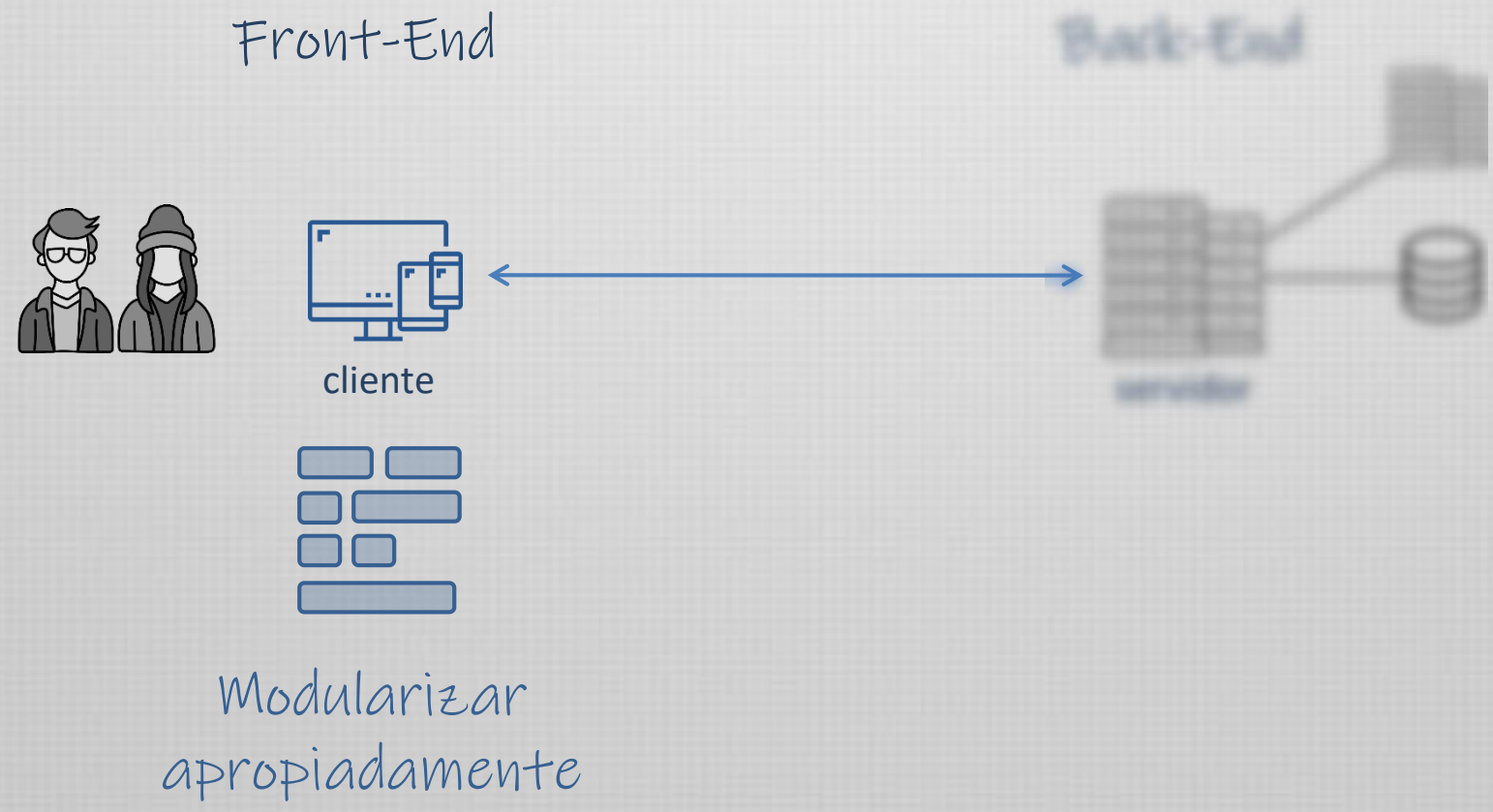
```
type StringOrNumber = string | number;
type ProcessStates = "open" | "closed";
type OddNumbersUnderTen = 1 | 3 | 5 | 7 | 9;
type AMessyUnion = "hello" | 156 | { error: true };
```

```typescript
interface ErrorHandling {
  success: boolean;
  error?: { message: string };
}

interface ArtworksData {
  artworks: { title: string }[];
}

interface ArtistsData {
  artists: { name: string }[];
}
```

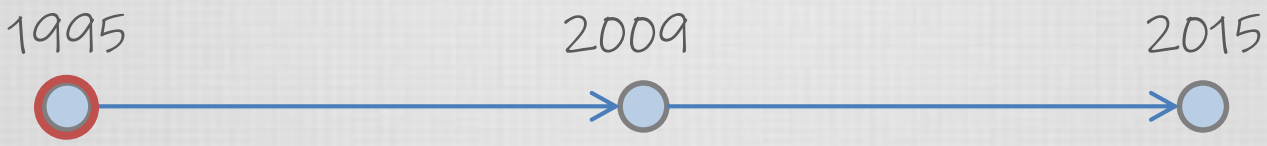
```typescript
type ArtworksResponse = ArtworksData & ErrorHandling;
type ArtistsResponse = ArtistsData & ErrorHandling;
```

```typescript
const handleArtistsResponse = (response: ArtistsResponse) =>
{
  if (response.error) {
    console.error(response.error.message);
    return;
  }

  console.log(response.artists);
};
```

Escenario web



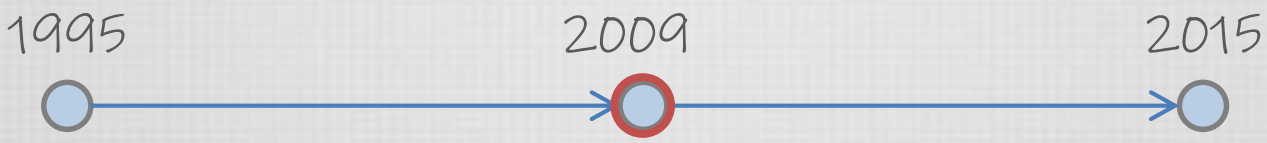
Módulos JavaScript



IIFE - Immediately Invoked Function Expression

```
const modulo = (function () {
  // variables
  // funciones
  return{
        // funciones exportadas
  }
})();
```

Módulos JavaScript



CommonJS Modules

OPCIÓN #1

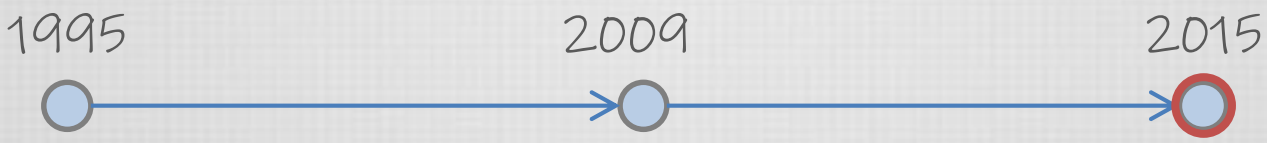
```
module.exports = {

        miFuncion : () {...}

};
```

OPCIÓN #2

```
function f() {...}

module.exports.miFuncion = f;
```

```
const moduloA = require('./file.js'):
...
moduloA.miFuncion()
```

Módulos JavaScript



ECMAScript Modules

```
function f(){...}
function g(){...}
export function h(){...}

export {f, g}
```

Named Exports

```
import {f,h} from './file.js';
...
```

```
function f(){...}

export default f
```

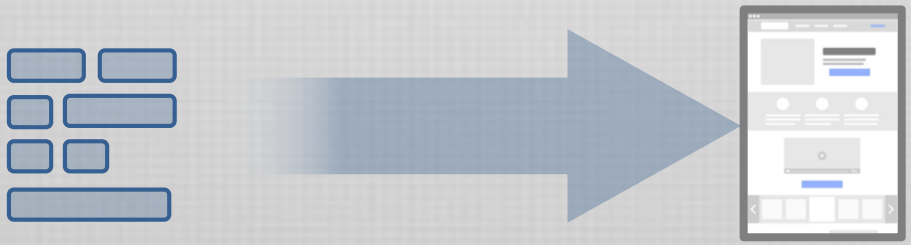
Default Export

```
import f from './file.js';
...
```

Modularización en la interfaz

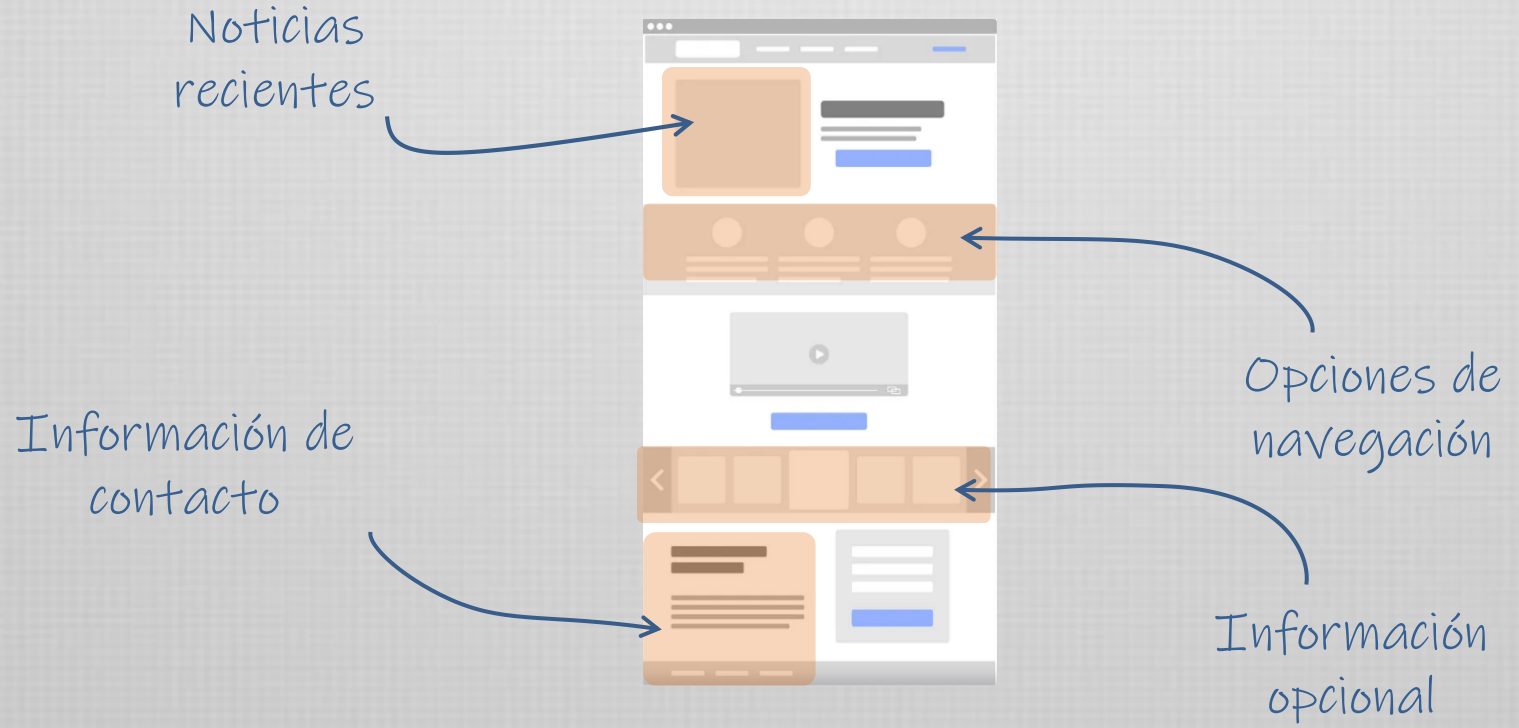
¿Como podemos conceptualizar una modularización en la interfaz?

Módulos JavaScript



Interfaz Web

Componentes

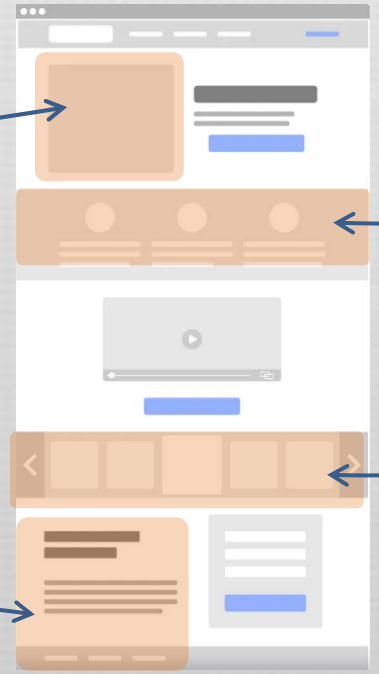


Componentes

"renderiza"

```
<div class="imagen">
 <button>ABC</button>
 <img src="img1.jpg">
 <div><h3>Imagen</h3></div>
</div>
```

<Noticias>



No son tags HTML

<Main-Nav>

<Contacto>

<Opcionales>

Componentes

Pagina.html

```
...
<div>...</div>
<div>
 <Contacto>
</div>
<p>
...
```

Script.js

```
...
elements = document.querySelector("Contacto");
var tag = elements[0];
var nuevocontenido = "Hola IAW";
var reemplazo = document.createTextNode(nuevocontenido);
tag.parentNode.replaceChild(replacement, tag);
...
```

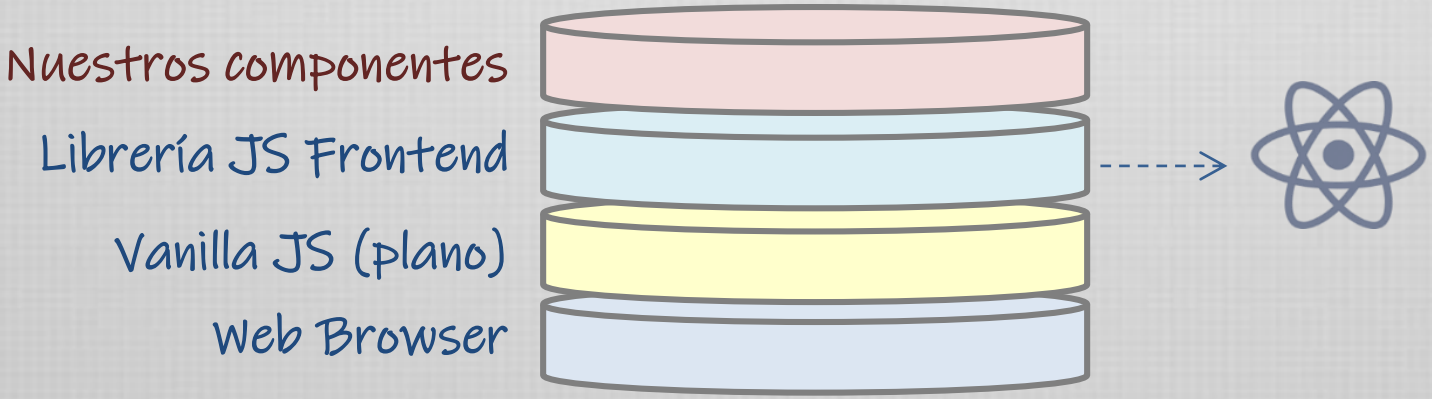
Contacto.js

```
export function Contacto()
        return(
        // contenido del elemento
        )
}
```

¿Y si usamos módulos?

Requiere una maquinaria que entienda de qué se trata esto

Componentes



Requiere una maquinaria que entienda de qué se trata esto

Librería de JavaScript

Nos provee funciones y objetos para construir Interfaces de Usuario

Basado en Componentes

Encapsulan datos y la forma de verlos (renderizarlos)

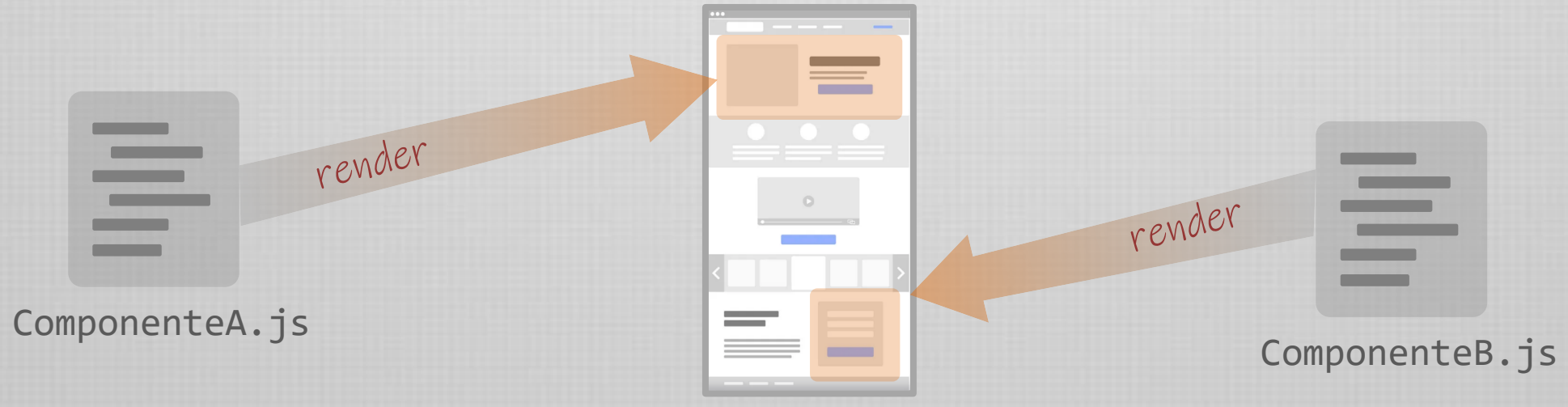
Declarativo

Nos enfocamos en cómo se ven los componentes

Puede ejecutarse en un servidor

Podemos "servir" páginas complejas hacia un navegador

React



Componentes

```
function MiComponente() {

    // datos
    // useState

    const funcionalidadX = () =>{
        // cambiamos los datos
    }

    return (
        // renderizamos en JSX
        <div>…</div>
     );
}

export default MiComponente;
```

MiComponente.js

```
class MiComponente extends React.Component {

    // datos

    const funcionalidadX = () =>{
        // cambiamos los datos
    }

    render() {
        return (// renderizamos en JSX
        <div>…</div>
        )
    };
}
export default MiComponente;
```

MiComponente.js

JavaScript Syntax Extension
JavaScript XML

Parece HTML, pero no lo es
Admite elementos procesables
Simplifica la combinación de JS+HTML

```
<div>
        {sections.map((n,i) => (
            <div key={"section-" + n}>
                Sección {n} {i === 0 && <span>(primera)</span>}
            </div>
        ))}
</div>
```

REGLAS

Devolver un elemento raíz siempre

```
<div>
…
</div>
```

```
<>
…
</>
```

El cierre de tags es obligatorio

```
<div>
…
</div>
```

```
<img src="…" />
<br/>
<li>Un item</li>
```

Atributos JSX ≠ Atributos JS

```
<div className="alerta">
…
</div>
```

JSX